邦布 AI 工程对接文档

> 版本：v0.3（MVP）
> 负责人：AI 工程
> 对接方：后端团队
> 状态：待后端确认
> 变更：合并 v0.2 + 六项结构性缺口修补

一、文档目的

明确 AI 工程（情绪识别 + 对话陪伴 Agent）与**后端**之间的数据接口边界、字段定义、失败处理与情绪数据规范。

本文档为 MVP 联调的**基准契约**。第十二节「待确认项」全部拍板后，本文档自动升级为**唯一依据**；在此之前，双方可按本文档先行开发，但涉及待确认项的字段以最终拍板结果为准。

注意：AI 工程**只与后端对接**，硬件不直连 AI 工程。

```
硬件 → 后端 → AI 工程 → 后端 → 硬件
```

二、职责边界

模块	负责方	说明
音频采集	硬件	麦克风采集、编码、上传
音频传输	硬件 → 后端	硬件将音频送达后端
ASR	AI 工程	Doubao-Seed-ASR-2.0；text 模式跳过
情绪识别	AI 工程	Doubao-Seed-2.0-Mini + Prompt
对话生成	AI 工程	Doubao-Seed-2.1-Turbo + Prompt
情绪事件存储	后端	AI 工程不直接写库，情绪事件的落库由后端负责；AI 工程仅通过 `/v1/chat` 响应的 `daily_report_delta` 回传
当天情绪报告	后端	AI 工程提供结构化数据，后端生成报告
跟进策略	AI 工程	待跟进事件统一通过 `GET /v1/emotion/events` 获取，后端不在 `/v1/chat` 请求体中传入
TTS	AI 工程	seed-tts-2.0，返回音频 URL
音频销毁	后端	播放完成后销毁整个语音文件

模块	负责方	说明
文本朗读	硬件	语音模块朗读
兜底话术存储	后端	AI 工程提供文案，后端存储

关键约定：

- AI 工程返回**音频 URL + 文本**，音频由后端销毁。
- persona 不在请求体中传递；AI 工程根据 `user_id` 通过后端接口 `GET /v1/persona?user_id=xxx` 查询，查不到时用出厂默认 persona 兜底，出厂默认 persona 由 AI 工程整理。
- App 打字对话与硬件语音对话**不算同一个 session**。

三、接口总览

接口	方向	用途
POST /v1/chat	后端 → AI 工程	主对话接口
POST /v1/emotion/analyze	后端 → AI 工程	单独情绪分析（可选）
GET /v1/persona	AI 工程 → 后端	查询用户人格配置
GET /v1/emotion/events	AI 工程 → 后端	查询待跟进情绪事件
GET /v1/health	后端 → AI 工程	健康检查

接口前缀统一为 `/v1/`。

AI 工程在情绪识别完成后，根据 `need_followup` 决定是否调用 `GET /v1/emotion/events` 查询待跟进事件；`need_followup=false` 时可不查。

四、主接口：POST /v1/chat

4.1 请求（后端 → AI 工程）

Content-Type： `application/json`

audio 模式示例

```
{
  "request_id": "req_20250101_0001",
  "user_id": "u_123456",
  "device_id": "d_654321",
  "session_id": "s_20250101_001",
  "session_type": "hardware",
```

```
  "timestamp": 1735689600000,
  "input_type": "audio",
  "audio": {
    "format": "wav",
    "sample_rate": 16000,
    "channels": 1,
    "duration_ms": 4200,
    "url": "https://cos.example.com/audio/xxx.wav?signed=xx
x",
    "size_bytes": 134400
  },
  "context": {
    "recent_dialogue": [
      {"role": "user", "text": "今天好累"},
      {"role": "assistant", "text": "辛苦啦，发生什么了？"}
    ]
  }
}
```

text 模式示例

```
{
  "request_id": "req_20250101_0002",
  "user_id": "u_123456",
  "device_id": "d_654321",
  "session_id": "s_20250101_002",
  "session_type": "app",
  "timestamp": 1735689605000,
  "input_type": "text",
  "text": "今天被领导批评了，好烦",
  "context": {
    "recent_dialogue": [
      {"role": "user", "text": "在吗"},
      {"role": "assistant", "text": "在的，怎么啦？"}
    ]
  }
}
```

字段说明补充

字段	类型	必填	说明
text	string	input_type=text 必填	文本输入，替代 audio 对象
audio	object	input_type=text 时省略	—

text 模式约定：

- `input_type=text` 时，`audio` 对象整体省略，`text` 字段必填。
- text 模式**跳过 ASR**，直接进入情绪识别 + 对话生成。
- 本示例采用 `session_type=app`，与硬件语音 session 不共享上下文；`session_type` 与 `input_type` 的通用关系见下方「session 定义」。

字段说明

字段	类型	必填	说明
request_id	string	是	请求唯一 ID，幂等用
user_id	string	是	用户唯一 ID，AI 工程据此查 persona
device_id	string	是	设备唯一 ID
session_id	string	是	会话 ID，见下方 session 定义
session_type	string	是	`hardware` / `app`，两者不共享 session
timestamp	int	是	毫秒时间戳，用后端时钟
input_type	string	是	`audio` / `text`
audio	object	input_type=audio 必填	音频信息
audio.format	string	是	wav / mp3 / opus
audio.sample_rate	int	是	建议 16000
audio.channels	int	是	1（单声道）
audio.duration_ms	int	是	音频时长
audio.url	string	是	带签名的临时 URL，有效期 ≥ 5 分钟
audio.size_bytes	int	否	音频体积
audio.base64	string	否	备选方式，体积 ≤ 5MB

字段	类型	必填	说明
text	string	input_type=text 必填	文本输入
context	object	否	上下文
context.recent_dialogue	array	否	最近 N 轮，建议 ≤ 10 轮

persona 不在请求体中传递，AI 工程通过 `GET /v1/persona?user_id=xxx` 查询，查不到时用出厂默认 persona 兜底。出厂默认 persona 由 AI 工程整理。

待跟进事件统一由 AI 工程调用 `GET /v1/emotion/events` 获取，后端不在 `/v1/chat` 请求体中传入。

session 定义

- **一次唤醒 = 一个 session**。用户唤醒设备后，到设备进入休眠（或超时无交互）为止，算一个 session。
- 同一个 session 内用户连续多轮对话，**共享** `session_id`，上下文连续。
- 设备休眠后再次唤醒，**生成新** `session_id`，上下文不跨 session 传递（待跟进事件统一通过 `GET /v1/emotion/events` 获取）。
- **App 打字对话与硬件语音对话不共享 session**，由 `session_type` 区分。
- session 超时时间：**60s 无交互自动结束**（后端负责计时与判定，AI 工程不感知超时）。
- **session_type 与 input_type 的关系**：
  - 不强制一一对应，允许交叉。
  - `session_type=hardware` 时，input_type 可为 `audio`（主流）或 `text`（硬件屏输入，预留）。
  - `session_type=app` 时，input_type 可为 `text`（主流）或 `audio`（App 录音，预留）。
  - **上下文隔离以** `session_type` **为准**，不以 `input_type` 为准。
  - MVP 阶段实际只用两种组合：`hardware + audio`、`app + text`。

音频传输约定

- 优先 URL 方式：后端先上传音频到对象存储，给 AI 工程**带签名的临时 URL**。
- URL 有效期：≥ 5 分钟（AI 工程下载完成后即失效，后端可设短有效期）。
- 备选：Base64 内嵌，体积 ≤ 5MB，字段名 `audio.base64`。

- 不接受：音频流式分片（MVP 阶段不做）。

persona 约定

- AI 工程**不接收** persona 字段，只收 `user_id`。
- AI 工程通过后端接口 `GET /v1/persona?user_id=xxx` 查询。
- 查不到 → 用出厂默认 persona 兜底。
- 出厂默认 persona 由 AI 工程整理，待整理完成前先用占位值。

4.2 响应（AI 工程 → 后端）

```
{
  "request_id": "req_20250101_0001",
  "session_id": "s_20250101_001",
  "reply_text": "听起来今天真的不容易，要不要跟我说说发生了什么？",
  "reply_audio_url": "https://cos.example.com/reply/xxx.mp
3?signed=xxx",
  "emotion": {
    "label": "tired",
    "label_cn": "疲惫",
    "intensity": 0.82,
    "need_followup": true,
    "followup_status": "pending",
    "evidence": "用户说今天好累"
  },
  "followup_used": false,
  "daily_report_delta": {
    "abnormal_events": [
      {
        "event_id": "e_20250101_0001",
        "label": "tired",
        "label_cn": "疲惫",
        "intensity": 0.82,
        "evidence": "用户说今天好累",
        "followup_status": "pending"
      }
    ],
    "user_status": "tired"
  },
```

```
  "fallback_used": false,
  "latency_ms": 2350,
  "error": null
}
```

字段说明

字段	类型	说明
request_id	string	原样返回
session_id	string	原样返回
reply_text	string	纯文本，不含 emoji / markdown / 特殊符号
reply_audio_url	string	TTS 音频 URL，后端播放后销毁
emotion	object	本轮情绪识别结果
emotion.label	string	情绪标签（英文枚举）
emotion.label_cn	string	中文标签
emotion.intensity	float	强度 0-1
emotion.need_followup	bool	是否需跟进，**随 intensity 动态判定**
emotion.followup_status	string	`pending` / `followed` / `resolved` / `still_abnormal`
emotion.evidence	string	简短依据
followup_used	bool	本轮是否使用历史情绪跟进
daily_report_delta	object	当天情绪报告的增量数据，只含 `abnormal_events` 和 `user_status`；`report_date` 由后端按自己服务器时区补全，AI 工程不回传
daily_report_delta.abnormal_events	array	异常事件列表（6.2 完整事件的子集）
daily_report_delta.user_status	string	用户当天整体状态；取值复用情绪标签（如 `tired` / `down` / `happy`），当天完全没有任何情绪事件时为 `normal`；在用户与硬件一次对话的整个过程中计算一次，多轮对话时进行轮次计算，覆盖上一轮值

字段	类型	说明
fallback_used	bool	本轮是否用了兜底话术
latency_ms	int	AI 工程内部耗时，包含音频下载时间、调后端查 persona 时间、调后端查待跟进事件时间；不包含后端与 AI 工程之间的网络往返、硬件播放时间
error	object/null	错误信息

daily_report_delta 与 6.2 的关系

- `daily_report_delta` 是**本轮对话产生的增量**，用于驱动后端生成/更新当天报告。
- 后端落库时，以 6.2 的完整 Schema 为准。
- `abnormal_events` 数组元素为**冻结字段清单**，仅含以下 6 个字段：

字段	类型	说明
event_id	string	事件唯一 ID，后端据此关联 6.2 完整记录
label	string	情绪标签
label_cn	string	中文标签
intensity	float	强度 0-1
evidence	string	简短依据
followup_status	string	与 6.2 同名同义；回传条件见下方「回传频率」

- **delta 不包含的字段**（user_id、session_id、timestamp、source、report_date）由后端**自行补全**：
  - user_id / session_id / timestamp：从本次请求体取。
  - source：**固定为** `chat`（MVP 阶段仅有对话来源，不预留其他值）。
  - `report_date`：由后端按自己服务器时区生成，AI 工程不参与、不回传。
- **回传频率**：
  - `daily_report_delta` **每轮对话都回传**，内容为增量。
  - `abnormal_events` 仅含**本轮新增或状态变更**的事件；本轮无变化时为空数组 `[]`。
  - 状态变更时，旧事件在 `abnormal_events` 里带完整 6 字段（`event_id`、`label`、`label_cn`、`intensity`、`evidence`、`followup_status`），不是只带 `event_id`。

- `followup_status` **仅在状态变更时回传**；状态未变时该字段在 `abnormal_events` 元素中省略。
- `user_status` 每轮都回传。
- `daily_report_delta` 只含 `abnormal_events` 和 `user_status`，不包含 `report_date`；`report_date` 由后端按自己服务器时区补全，AI 工程不回传。

五、辅助接口

5.1 POST /v1/emotion/analyze（可选）

`/v1/chat` 内部调用 `/v1/emotion/analyze` 对应的情绪分析函数，`/v1/emotion/analyze` 作为独立接口对外保留。

请求

```
{
  "user_id": "u_123456",
  "session_id": "s_20250101_001",
  "input_type": "text",
  "text": "今天被领导批评了，好烦"
}
```

响应

```
{
  "emotion": {
    "label": "down",
    "label_cn": "低落",
    "intensity": 0.78,
    "need_followup": true,
    "followup_status": "pending",
    "evidence": "被领导批评"
  }
}
```

字段说明

字段	类型	说明
emotion.label	string	情绪标签

字段	类型	说明
emotion.label_cn	string	中文标签
emotion.intensity	float	强度 0-1
emotion.need_followup	bool	是否需跟进
emotion.followup_status	string	pending / followed / resolved / still_abnormal
emotion.evidence	string	简短依据

5.2 GET /v1/persona

请求： `GET /v1/persona?user_id=u_123456`

响应

```
{
  "persona": {
    "name": "邦布",
    "gender": "female",
    "personality": "温柔、耐心",
    "speaking_style": "轻松、口语化",
    "relationship": "朋友",
    "custom_keywords": ["别太啰嗦", "多鼓励我"]
  }
}
```

说明：查不到时返回默认 persona。

5.3 GET /v1/emotion/events

待跟进事件统一由 AI 工程调用本接口获取，后端不在 `/v1/chat` 请求体中传入。

AI 工程在情绪识别完成后，根据 `need_followup` 决定是否调用本接口；`need_followup=false` 时可不查。

请求

```
GET /v1/emotion/events?user_id=u_123456&followup_status=pending&report_date=2025-01-01
&limit=5&offset=0
```

Query 参数说明

参数	类型	必填	说明
user_id	string	是	用户唯一 ID
followup_status	string	否	默认 `pending`；可选 `followed` / `still_abnormal`
report_date	string	是	YYYY-MM-DD，只查当天
limit	int	否	默认 5，最大 20
offset	int	否	默认 0，分页用

响应

```
{
  "events": [
    {
      "event_id": "e_20250101_0001",
      "timestamp": 1735689600000,
      "label": "tired",
      "intensity": 0.82,
      "evidence": "用户说今天好累"
    }
  ],
  "pagination": {
    "total": 12,
    "limit": 5,
    "offset": 0,
    "has_more": true
  }
}
```

分页字段说明

字段	类型	说明
pagination.total	int	当天符合条件的事件总数
pagination.limit	int	本次请求的 limit
pagination.offset	int	本次请求的 offset
pagination.has_more	bool	是否还有下一页

说明：

- 只查**当天**的待跟进事件（`report_date` 必填）。

- 支持 `limit` 和 `offset` 分页。

5.4 GET /v1/health

```
{
  "status": "ok",
  "asr": "ok",
  "llm": "ok",
  "tts": "ok",
  "version": "0.3.0"
}
```

六、情绪标签体系（核心）

6.1 标签枚举

label	中文	典型触发	need_followup
happy	开心	兴奋、分享好事	false
calm	平静	中性陈述（含“今天周三”等）	false
neutral	中性	无法判断情绪倾向	false
tired	疲惫	累、困、没力气	动态
anxious	焦虑	担心、紧张、压力	动态
down	低落	难过、失落、被批评	动态
angry	生气	愤怒、抱怨、被冒犯	动态
bored	无聊	没事做、没意思	false

规则

- MVP 只取主标签，不做混合情绪。
- **calm 与 neutral 的判定优先级**：
  - 文本**存在明确的中性陈述内容**（如“今天周三”“我要喝水”）→ `calm`
  - 文本**无法判断情绪倾向且无明确中性陈述**（如语气词、碎片化输入）→ `neutral`
  - 判定顺序：先判断是否命中 `happy / tired / anxious / down / angry / bored` 六类；未命中且为完整陈述 → `calm`；未命中且为碎片/无法判断 → `neutral`
- **情绪异常类标签**（闭集）：`tired` / `anxious` / `down` / `angry`

- **非异常类标签**：`happy` / `calm` / `neutral` / `bored`
- `need_followup` 判定公式：`need_followup = (label ∈ 异常类) AND (intensity ≥ 0.6)`
- 标签为闭集，新增需双方确认。

6.2 情绪事件 Schema（后端存储）

AI 工程在响应中通过 `daily_report_delta` 回传结构化数据，后端负责写库。`report_date` 由后端按服务器时区生成，AI 工程不参与。

```
{
  "event_id": "e_20250101_0001",
  "user_id": "u_123456",
  "session_id": "s_20250101_001",
  "timestamp": 1735689600000,
  "label": "tired",
  "label_cn": "疲惫",
  "intensity": 0.82,
  "need_followup": true,
  "followup_status": "pending",
  "evidence": "用户说今天好累",
  "source": "chat",
  "report_date": "2025-01-01"
}
```

状态流转

```
pending → followed → resolved
              ↓
        still_abnormal → 每天 24:00（服务器时区）定时批量归档
```

状态	含义	更新方	归档规则
pending	待跟进	AI 工程回传，后端落库	—
followed	已跟进	AI 工程	—
resolved	已缓解	AI 工程	—
still_abnormal	仍异常	AI 工程	**后端每天 24:00（服务器时区）定时批量归档，不进入次日 pending 池**

关键规则：

- 情绪跟进**只限当天**。
- 当天未恢复 → 写进报告，**不跨天跟进**。
- `followup_status` 的更新（`pending → followed → resolved / still_abnormal`）由 AI 工程在 `/v1/chat` 响应的 `daily_report_delta` 中回传，后端负责落库；归档由后端每天 24:00（服务器时区）执行。
- **still_abnormal 的更新方是 AI 工程**：当 AI 工程在跟进对话中判断用户情绪未恢复时，在响应中将该事件的 `followup_status` 置为 `still_abnormal`。
- `still_abnormal` 和 `resolved` 的判定由 AI 工程用规则判断，不是 LLM 判断。
- 判定是多轮的，直到 AI 判断没问题为止；情绪跟进仍只限当天，不跨天跟进。
- 如果 AI 跟进了但无法判断，保持 `followed`。
- **归档方是后端，触发条件为「24:00 定时批量归档」**：
  - 后端在每天 24:00（服务器时区）对当天所有 `still_abnormal` 事件统一归档进当天报告。
  - **不在每次 chat 响应时检查并归档**，避免归档时机不确定。
  - 当天最后一次对话结束不触发归档，仅 24:00 触发。
- `still_abnormal` **不进入次日 pending 池**。
- 若产品侧需要「当天实时完善报告」，由后端在每次 chat 响应后**更新报告内容**，但**归档动作仍在 24:00 执行**。

七、错误处理

7.1 错误码

code	含义	建议处理
400	参数错误	后端修正后重试
401	鉴权失败	检查 API Key
413	音频过大	压缩后重传
422	音频格式不支持	转码后重传
429	模型限流	退避重试
500	AI 工程内部错误	重试 1 次
503	模型服务不可用	降级兜底

code	含义	建议处理
504	超时	重试或兜底

7.2 错误响应结构

```
{
  "request_id": "req_20250101_0001",
  "error": {
    "code": 503,
    "message": "LLM service unavailable",
    "retryable": true
  }
}
```

7.3 兜底策略

兜底话术由 AI 工程提供文案，后端存储，硬件播放。

音频与文本优先级：

- `reply_audio_url` 有值且 `fallback_used=false` 时，硬件优先播放音频。
- `reply_audio_url` 为空，或 `fallback_used=true` 时，硬件朗读 `reply_text`。
- `reply_audio_url` 有值但播放失败：后端自动重发一次；重发仍失败，硬件朗读 `reply_text`，并计 1 次失败。

场景	兜底话术
AI 工程超时	“我有点没听清，你再说一次好吗？”
ASR 失败	“我有点没听清，你再说一次好吗？”
LLM 答不出	“我现在有点累，你晚点再问我可以吗？”
LLM 内容不合适	“我现在有点累，你晚点再问我可以吗？”
TTS 失败	返回文本，硬件直接朗读文本
硬件没收到音频	后端自动重发一次；重发仍失败，硬件朗读 `reply_text`，并计 1 次失败

失败计数规则：

- 同一个 session 内，以下事件**累计**计数：
  - ASR 失败
  - LLM 失败（含答不出、内容不合适）

    - TTS 失败
    - 后端未收到 AI 工程响应（超时）
- **「硬件未收到音频」不计入失败计数**，由后端独立处理：后端自动重发一次；重发仍失败，才计 1 次失败。
- 累计 3 次 → 进入静默。
- 用户下次唤醒时，计数清零。

八、性能与超时约定

指标	约定值
端到端超时（后端 → AI 工程）	≤ 30s
AI 工程内部目标耗时	≤ 5s（音频 ≤ 10s）
ASR 环节耗时预算	≤ 2s
情绪识别耗时预算	≤ 1s
对话生成耗时预算	≤ 3s
TTS 耗时预算	≤ 2s
音频最大时长	≤ 60s
音频最大体积	≤ 10MB
并发（MVP）	≤ 10 QPS
TTS RPM 限制	10
ASR RPM 限制	100

耗时说明：AI 工程内部 ≤ 5s 为各环节总和上限；端到端 30s 含后端与 AI 工程之间的网络往返、音频下载、重试余量。

九、幂等与重试

- 同一 `request_id` 重复请求，AI 工程返回缓存结果（缓存 5 分钟）。
- 缓存 key 为 `request_id`，缓存内容为完整响应。
- MVP 阶段用内存字典实现，5 分钟过期。
- 后端重试建议：最多 1 次，间隔 2s。
- 同一 `session_id` 的并发请求，AI 工程串行处理。

十、鉴权

- 方式：API Key（Header：`X-API-Key`）。
- **由 AI 工程颁发给后端**，硬件不持有。
- API Key 由 AI 工程生成后线下告知后端，后端在 Header 中带 `X-API-Key` 调用 AI 工程。
- MVP 阶段单 Key，后续可升级为多 Key + 权限。

十一、音频与隐私

- `audio.url` 为**带签名的临时 URL**，有效期 ≥ 5 分钟，不公开。
- `reply_audio_url` 为 TTS 输出音频，后端播放完成后**销毁整个语音文件**。
- `reply_audio_url` 的有效期从 AI 工程返回起，到后端播放完成为止；如果后端播放失败，URL 保留到播放成功为止。
- 销毁由后端程序执行，不存在提前销毁问题。
- 硬件未收到音频 → 后端自动重发一次；重发仍失败，才计 1 次失败（见 7.3 节）。

十二、待确认项

#	事项	决定方
1	音频传输方式：URL / Base64	后端
2	音频格式与采样率最终值	硬件
3	默认 persona 具体值	AI 工程
4	情绪标签映射硬件表情	硬件 + 后端
5	API Key 分发方式（已定为线下告知）	AI 工程
6	是否允许混合情绪（待定）	双方
7	跟进频率上限（待定）	产品+AI
8	当天情绪报告具体字段	后端+AI

已定事项：session 超时时间已定为 **60s 无交互自动结束**，由后端负责计时与判定，不列入以上 8 项待确认事项。

生效状态与暂定字段处理：

- 截至 v0.3，本节第 5 项 API Key 分发方式**已定为线下告知**；其余 7 条尚未形成正式结论（第 3 项为「待整理」，第 6、7 项保留为待定）。按本节定义，本文档当前为「可先行开发的基准契约」，**尚未生效为联调唯一依据**。

- 任一条未决时，涉及该条的接口字段按以下规则处理：
  - 后端按本文档字段表中的**示例值**构造请求。
  - AI 工程按本文档字段表中的**示例值**返回响应。
  - 双方不得擅自变更字段名、类型。
  - 若示例值不可用，双方临时约定，记入变更记录，待正式拍板后回填。
- **生效条件**：本节 8 条全部填写正式「结论」后，本文档自动升级为联调唯一依据，暂定字段以拍板结果为准。

十三、变更记录

版本	日期	变更	负责人
v0.1	2025-10-08	初稿	AI 工程
v0.2	2025-10-08	修订阻塞项、歧义项、安全项；合并用户拍板决策	AI 工程
v0.3	2025-10-08	修补六项结构性缺口：生效条件、text 示例、Schema 对齐、still_abnormal 枚举、session 定义、变更日期	AI 工程